

SENAI · IA PARA DEVs

Dia 1 — DevOps Inteligente e Detecção de Falhas

CI/CD end-to-end, diagnóstico assistido por IA e validação pós-deployment

Guia prático do aluno

Professor: Wanderson Souza

Versão consolidada · Agosto de 2026

Repositório-base: IA-para-DEVs-SCTEC-T2/pipeline-guardian-agent-demo

Apresentação do laboratório

Neste laboratório, você acompanhará uma aplicação desde a alteração do código até a validação da versão publicada. O objetivo não é apenas “fazer o pipeline rodar”, mas compreender onde cada falha acontece, qual evidência comprova o problema e quem realmente decide pela aprovação ou reprovação da entrega.

Ao longo da atividade, você executará cinco cenários: uma entrega saudável, uma falha de lint, uma falha de testes, uma falha na construção da imagem Docker e uma falha funcional identificada somente depois do deployment.

Regra principal do laboratório

A IA explica os resultados. Os gates técnicos decidem.

Nenhuma recomendação do modelo substitui exit codes, status dos jobs, correlação de commit ou códigos HTTP.

Resultados de aprendizagem

- Distinguir integração contínua (CI) de entrega e validação pós-deployment (CD).
- Interpretar logs de lint, testes, build, Docker, plataforma e runtime.
- Entender a diferença entre processo saudável e funcionalidade saudável.
- Relacionar uma evidência externa, como HTTP 500, a um erro interno da aplicação.
- Compreender por que o diagnóstico por IA deve ser informativo, e não decisório.
- Operar branches temporárias, Pull Requests, GitHub Actions e environments do Railway com segurança.

Pré-requisitos

Recurso	Requisito
Node.js	Versão 22
Git e GitHub	Conta com permissão para criar branches e Pull Requests
Docker	Docker Engine ou Docker Desktop em funcionamento
Railway	Conta vinculada ao GitHub
Terminal	Bash ou equivalente
Ferramentas auxiliares	`jq` recomendado para formatar JSON

Atenção antes de começar

Faça os testes em um fork ou em um repositório de laboratório. Nunca aplique uma falha proposital diretamente na `main` do repositório oficial. As branches dos cenários 2 a 5 não devem ser integradas.

Sumário do material

1. Prompt para preparar o repositório
2. Arquitetura e responsabilidades
3. Validação local da estrutura
4. Configuração do GitHub

5. Configuração do Railway em produção
6. Regra de branches para os cenários
7. Cenário 1 — execução saudável
8. Cenário 2 — falha de lint
9. Cenário 3 — falha de testes
10. Cenário 4 — falha no Docker build
11. Cenário 5 — falha funcional pós-deployment
12. Fixtures offline e plano de contingência
13. Solução de problemas
14. Checklist final e síntese
15. Desafio em grupos — implantação e investigação cruzada

1. Prompt para preparar o repositório

Esta etapa prepara toda a infraestrutura do laboratório. O prompt deve ser executado em uma cópia controlada do repositório, preferencialmente por quem estiver preparando o ambiente da turma. Leia o resultado, valide os arquivos alterados e execute todos os comandos de aceite antes de criar uma Pull Request.

Sugestão de uso

Para a primeira transformação estrutural, utilize um modelo de alta capacidade e raciocínio elevado. Depois que a base estiver pronta, reduza o nível de esforço para correções pontuais e validações.

Prompt consolidado

Você é um engenheiro de software sênior especializado em Node.js, Express, React/Vite, npm workspaces, Docker, GitHub Actions, Railway, observabilidade, segurança de pipelines e integração com a OpenAI.

Trabalhe diretamente no repositório atual:
IA-para-DEVs-SCTEC-T2/pipeline-guardian-agent-demo

OBJETIVO

Adaptar o repositório para o Dia 1 da semana “DevOps Inteligente e Detecção de Falhas”, criando um laboratório completo e didático de CI/CD end-to-end com:

- aplicação full stack executável localmente e em produção;
- container de produção único;
- CI com lint, testes, build e Docker build;
- diagnóstico automatizado dos logs;
- deployment no Railway pela integração nativa com GitHub;
- smoke test real pós-deployment;
- coleta best-effort dos logs internos do Railway;
- relatório operacional em JSON, Markdown e HTML;
- cinco cenários reproduzíveis;
- gates técnicos determinísticos e independentes da IA.

RESTRIÇÕES IMPORTANTES

1. Preserve a aplicação CopaFigurinhas e os workspaces backend, frontend e automation.
2. Use Node.js 22 e JavaScript ESM.
3. Não adicione banco, Redis, migrations, SQLite, Kubernetes ou Terraform.
4. Mantenha os dados em memória.
5. Não transforme o projeto em TypeScript.
6. Não faça commit, push, merge ou Pull Request.
7. Não exponha secrets em logs, relatórios, artifacts ou summaries.

8. A IA nunca pode aprovar ou reprovar CI/CD; ela apenas explica fatos técnicos.
9. Sem OPENAI_API_KEY, tudo deve funcionar com classificador determinístico.
10. Preserve validação Zod, redaction de segredos e diagnósticos ancorados.

ARQUITETURA DE PRODUÇÃO

Implemente um único container:

copa-figurinhas

- └ Express servindo a API em /api
- └ frontend/dist servido pelo mesmo Express

Requisitos:

- desenvolvimento com frontend e backend separados;
- produção com Express servindo a SPA;
- refresh de rotas da SPA funcionando;
- /api inexistente retorna JSON 404, nunca index.html;
- preservar ou criar /api/health e /api/version;
- respeitar process.env.PORT;
- priorizar RAILWAY_GIT_COMMIT_SHA para identificar a release;
- logs estruturados JSON em stdout;
- incluir requestId, service, environment, version, commitSha, eventType, phase, method, path, statusCode e durationMs;
- registrar functional.report.completed e functional.report.failed.

CONTAINER E RAILWAY

Criar ou ajustar Dockerfile multi-stage, .dockerignore, compose.yaml, railway.json e documentação.

O Dockerfile deve:

- usar npm ci;
- compilar o frontend no estágio de build;
- instalar somente dependências de produção do backend no runtime;
- copiar backend/src e frontend/dist;
- executar como usuário não root;
- iniciar com node backend/src/server.js;
- ter health check sem depender de curl ou wget.

O railway.json deve declarar Dockerfile, /api/health, timeout e restart policy.
Não cadastrar OPENAI_API_KEY no Railway.

CI — .github/workflows/ci.yml

Acionamentos:

- pull_request para main;
- push para main.

Jobs paralelos:

1. quality: npm ci, npm run lint, reports/lint.log, artifact lint-log.
2. tests: npm ci, npm run test, reports/tests.log, artifact tests-log.
3. build: npm ci, npm run build, reports/build.log, artifact build-log.
4. docker-build: docker build --progress=plain, reports/docker.log, artifact docker-log, sem publicar imagem.
5. diagnose: needs dos quatro jobs, if always, continue-on-error, coleta best-effort, diff da PR, diagnóstico e relatório em JSON/MD/HTML.
6. ci-gate: decide somente pelos quatro resultados técnicos; não depende da IA.

O relatório deve distinguir SUCCESS, FAILURE, SKIPPED, NOT REACHED e UNKNOWN.
A primeira falha é escolhida por regra e registrada em firstFailedStage.

OPENAI

Usar Responses API com saída estruturada validada por Zod.
Padrões: gpt-5-mini, reasoning low, 2000 tokens, timeout 45 s, maxRetries 1 e store false.

Separar fatos, inferências, limitações de coleta, limitações do modelo, causa provável e recomendações.

A política determinística usa somente status, exit codes, códigos HTTP, correlação de commit e regras de segurança.

Registrar telemetria não sensível: modelo, response ID, latência, tokens, categoria de erro e usedFallback.

O sanitizer não pode mascarar métricas como "Tokens: 3.785", mas deve mascarar sk-*, ghp_*, github_pat_*, Bearer, api_key, access_token e credenciais.

POST-DEPLOY — .github/workflows/post-deploy.yml

Acionamentos:

- workflow_run depois do CI da main;
- workflow_dispatch com ref, railway_service e railway_environment.

O workflow não implanta. Ele valida o que o Railway já publicou.

Jobs:

1. resolve-context: resolve commit e run_id do CI.
2. collect-ci-evidence: baixa artifacts do run correto, best-effort.
3. post-deploy-smoke-test: valida APP_BASE_URL, /api/health, commit e /api/report.
4. collect-railway-evidence: coleta build, deploy e runtime pelo deployment ID.
5. operational-deployment-diagnosis: explica, sem decidir.
6. publish-operational-report: publica JSON, Markdown e HTML.
7. cd-gate: depende somente do smoke test.

SEGURANÇA

- não usar pull_request_target;
- permissões mínimas;
- checkout sem persistir credenciais;
- não executar artifacts, diffs ou logs;
- secrets somente nos jobs necessários;
- não imprimir log bruto do Railway com tee;
- publicar somente material sanitizado.

CINCO CENÁRIOS

1. success: tudo aprovado; firstFailedStage null.
2. ci-lint-failure: variável não utilizada; firstFailedStage ci_quality.
3. ci-tests-failure: expectativa incorreta; firstFailedStage ci_tests.
4. ci-docker-failure: COPY inexistente; firstFailedStage ci_docker.
5. cd-functional-failure: health 200, /api/report 500; firstFailedStage functional; smoke e CD gate falham.

Criar fixtures offline:

```
npm run pipeline:fixture -- success
npm run pipeline:fixture -- ci-lint-failure
npm run pipeline:fixture -- ci-tests-failure
npm run pipeline:fixture -- ci-docker-failure
npm run pipeline:fixture -- cd-functional-failure
```

Criar documentação:

- docs/day-1-lab.md;
- docs/day-1-failure-scenarios.md;
- docs/day-1-operational-logs.md;
- docs/day-1-evidence-review.md;
- docs/railway-setup.md;
- README e .env.example atualizados.

TESTES E ACEITE

```
npm run test:automation
npm run lint
npm run test
npm run build
npm run ci
docker build -t copa-figurinhas:day1 .
docker compose up --build
```

Teste OpenAI somente com opt-in:

```
OPENAI_LIVE_TEST=true npm run openai:smoke
```

Teste de fallback:

```
OPENAI_API_KEY= npm run deployment:analyze -- \
--log samples/deployment/startup-failure.log \
--status failure
```

Não use env -u OPENAI_API_KEY para provar fallback: o dotenv pode ler o .env.

ENTREGA

- listar arquivos criados e alterados;
- explicar decisões;
- informar comandos e exit codes;
- declarar o que não foi testado externamente;
- confirmar que nenhum secret foi versionado;
- não fazer commit, push ou PR.

Como validar a entrega do assistente de código

Não aceite apenas um resumo dizendo que “está tudo pronto”. Confira o diff, procure alterações inesperadas e execute os comandos de aceite. O resultado precisa ser reproduzível no seu computador antes de ser enviado ao GitHub.

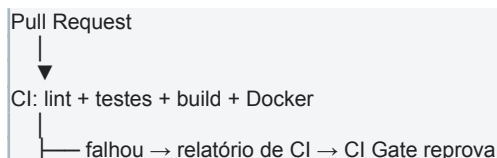
```
npm run test:automation
npm run lint
npm run test
npm run build
npm run ci
docker build -t copa-figurinhas:day1 .
docker compose up --build
```

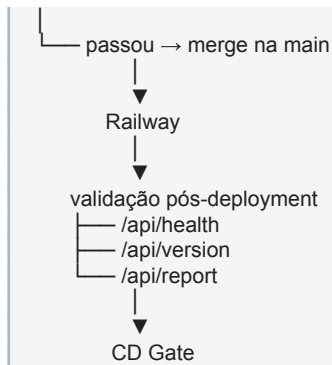
2. Arquitetura e responsabilidades

O laboratório foi desenhado para evitar uma confusão comum: atribuir ao diagnóstico por IA o poder de decidir se uma entrega está correta. Aqui, cada componente possui uma responsabilidade explícita.

Componente	Responsabilidade	Pode reprovar?
GitHub Actions — CI	Executar lint, testes, build e Docker build.	Sim, por regra técnica.
Pipeline Guardian	Ler evidências e explicar a falha.	Não.
Railway	Construir, implantar e executar o container.	A plataforma pode rejeitar uma instância não saudável.
Smoke test	Testar a aplicação publicada por HTTP.	Sim.
CD Gate	Converter o resultado do smoke test em aprovação ou reprovação.	Sim, deterministicamente.
OpenAI ou fallback	Gerar explicação, causa provável, limitações e ações.	Não.

Fluxo da entrega





Ideia-chave

Um deployment pode subir corretamente, responder ao health check e ainda assim apresentar uma falha de negócio. Por esse motivo, o laboratório valida uma rota funcional além de verificar se o processo está vivo.

3. Validação local da estrutura

Antes de criar branches de demonstração, confirme que a `main` está saudável. Uma falha preexistente altera a leitura dos cenários e impede que você identifique corretamente a primeira etapa que falhou.

```

git switch main
git pull --ff-only
git status --short

npm run lint
npm run test
npm run build
npm run ci
docker build -t copa-figurinhas:baseline .
  
```

Suba o mesmo container utilizado em produção:

```
docker compose up --build
```

Em outro terminal:

```

curl -i http://localhost:3001/api/health
curl -s http://localhost:3001/api/version | jq
curl -i http://localhost:3001/api/report
  
```

Finalize:

```
docker compose down
```

Checkpoint

A `main` só está pronta para os cenários quando lint, testes, build, CI e Docker passam; `/api/health` e `/api/report` respondem HTTP 200; e `git status --short` não mostra alterações.

4. Configuração do GitHub

4.1 Habilitar o GitHub Actions

```
Settings → Actions → General
```

Confirme que o repositório permite executar workflows. Ao abrir uma Pull Request, o workflow `CI` deve iniciar automaticamente.

4.2 Secrets

Settings → Secrets and variables → Actions → Secrets

Secret	Uso	Obrigatório?
OPENAI_API_KEY	Diagnóstico com modelo.	Não. Sem ela, o fallback funciona.
RAILWAY_TOKEN	Coleta de logs internos do environment selecionado.	Não para o gate; necessário para cobertura interna.

Segurança

Um Project Token do Railway é restrito a um environment. Um token de `production` não deve ser usado como se tivesse acesso ao `staging-lab`. No cenário 5, use um token criado especificamente para o staging.

4.3 Variables

Settings → Secrets and variables → Actions → Variables

Variável	Exemplo	Finalidade
APP_BASE_URL	https://dominio.up.railway.app	URL testada pelo smoke test.
OPENAI_MODEL	gpt-5-mini	Modelo do diagnóstico.
OPENAI_REASONING_EFFORT	low	Esforço de raciocínio.
OPENAI_MAX_OUTPUT_TOKENS	2000	Limite de saída.
RAILWAY_PROJECT	nome ou ID do projeto	Vínculo não interativo do coletor.
RAILWAY_SERVICE	copa-figurinhas	Serviço consultado.
RAILWAY_ENVIRONMENT	production	Environment padrão.

Somente `APP\_BASE\_URL` é necessária para o smoke test. As demais possuem padrões ou são usadas para enriquecer a coleta e o diagnóstico.

4.4 Como ler os jobs do CI

Job	O que verifica
Quality (lint)	Regras estáticas e padrões do código.
Tests	Comportamento esperado da aplicação.
Build	Compilação do frontend.
Docker build	Construção do artefato que será implantado.
Diagnose	Explicação das evidências; não é gate.
CI Gate	Decisão final baseada nos quatro jobs técnicos.

4.5 Preparação do repositório para a aula

Antes de cadastrar secrets ou variables, confirme que você está no repositório de laboratório correto e que a `main` está saudável. O objetivo é evitar que uma configuração de aula seja aplicada no repositório errado ou que uma falha preexistente seja confundida com um dos cenários.

1. Abra o repositório `pipeline-guardian-agent-demo` no GitHub.

2. Confirme que a branch padrão é `main` e que o arquivo `.github/workflows/ci.yml` existe.
3. Abra a aba Actions e verifique se o workflow `CI` aparece na lista de workflows.
4. Se houver uma execução anterior da `main`, confirme que ela está verde antes de iniciar os cenários.
5. Não configure credenciais em arquivos `.env` versionados, no README ou em comentários da Pull Request.

Checkpoint do repositório

O ponto de partida correto é: `main` saudável, workflow `CI` visível em Actions e nenhum secret exposto no código ou na documentação.

4.6 Permissões do GitHub Actions

Abra:

Settings → Actions → General

Revise dois grupos de configuração:

Configuração	O que conferir	Por que importa
Actions permissions	O repositório permite executar as actions utilizadas pelo workflow.	Sem essa permissão, o workflow pode ser bloqueado antes mesmo de executar o código.
Workflow permissions	O `GITHUB_TOKEN` possui a escrita necessária para o job de diagnóstico publicar ou atualizar o comentário da Pull Request.	O YAML continua declarando permissões mínimas, mas a configuração do repositório não pode impedir a operação.
Pull Requests	O fluxo da aula permite abrir PRs para `main` sem push direto.	Os cenários devem ser observados no CI sem integrar falhas intencionais.

O workflow não deve usar `pull\_request\_target` para executar código de branches de demonstração. Os secrets também não devem ser impressos em logs ou summaries.

4.7 Cadastrar os Secrets passo a passo

Abra:

Settings → Secrets and variables → Actions → Secrets → New repository secret

Cadastre cada secret separadamente. O GitHub exibirá apenas o nome depois de salvo; o valor não deve aparecer novamente em tela, logs ou documentação.

Nome	Como obter	Uso no laboratório
OPENAI_API_KEY	Chave da conta utilizada para o diagnóstico com modelo.	Opcional. Sem ela, o Pipeline Guardian utiliza fallback determinístico.
RAILWAY_TOKEN	Project Token criado no Railway para o environment que será consultado.	Usado para coleta de evidências internas do Railway. O token precisa pertencer ao environment correto.

Ao trocar de `production` para `staging-lab` no cenário 5, substitua temporariamente `RAILWAY\_TOKEN` pelo token criado especificamente para o staging. Ao encerrar a demonstração, restaure o token de produção.

Segurança dos Secrets

Nunca copie o valor de `OPENAI\_API\_KEY` ou `RAILWAY\_TOKEN` para Variables. Secrets são para credenciais; Variables são para configuração não sensível.

4.8 Cadastrar as Variables passo a passo

Abra:

Settings → Secrets and variables → Actions → Variables → New repository variable

Cadastre uma variável por vez. Use os nomes exatamente como aparecem no workflow e evite espaços adicionais ou aspas desnecessárias.

Variable	Preenchimento recomendado
APP_BASE_URL	URL pública completa do serviço, incluindo `https://`. Ex.: `https://seu-dominio.up.railway.app`.
OPENAI_MODEL	`gpt-5-mini` ou o modelo definido para a aula.
OPENAI_REASONING_EFFORT	`low` no laboratório.
OPENAI_MAX_OUTPUT_TOKENS	`2000`.
RAILWAY_PROJECT	Nome ou ID do projeto Railway.
RAILWAY_SERVICE	Nome exato do serviço. Ex.: `copa-figurinhas`.
RAILWAY_ENVIRONMENT	`production` durante o fluxo normal.

Após cadastrar `APP\_BASE\_URL`, abra a URL no navegador ou valide com `curl`. Uma URL sem protocolo, com domínio antigo ou apontando para o staging incorreto pode fazer o smoke test validar o ambiente errado.

4.9 Validar a primeira execução do CI no GitHub

Faça uma alteração pequena e legítima em uma branch de teste, abra uma Pull Request para `main` e acompanhe:

Actions → CI → execução da Pull Request

Job	Resultado esperado na baseline	O que observar
Quality	SUCCESS	`npm run lint` e artifact de lint.
Tests	SUCCESS	`npm run test` e artifact de testes.
Build	SUCCESS	`npm run build` e artifact de build.
Docker build	SUCCESS	Construção da imagem sem publicação.
Diagnose	SUCCESS ou fallback válido	Summary com explicação e artifacts de diagnóstico. Não é gate.

CI Gate

SUCCESS

Decisão baseada somente nos quatro jobs técnicos.

Dentro da execução, mostre aos alunos a aba Summary e a seção Artifacts. O Summary é o local onde o relatório Markdown do Pipeline Guardian é apresentado; os artifacts preservam logs e relatórios para inspeção posterior.

Regra didática

Se `Diagnose` produzir uma explicação diferente, mas os resultados técnicos forem os mesmos, o `CI Gate` não deve mudar. A IA explica; os gates técnicos decidem.

5. Configuração do Railway em produção

1. Entre no Railway com sua conta do GitHub.
2. Crie um projeto a partir do repositório `pipeline-guardian-agent-demo`.
3. Caso o repositório da organização não apareça, use `Configure GitHub App` e autorize apenas o repositório necessário.
4. No serviço, confirme branch `main`, raiz vazia, builder Dockerfile e health check `/api/health`.
5. Gere um domínio público em Settings → Networking → Public Networking.
6. Quando disponível, ative Wait for CI em produção.

5.1 Variáveis que não devem ser cadastradas

Variável	Por que não cadastrar
PORT	O Railway injeta a porta automaticamente.
NODE_ENV	A imagem já executa em produção.
COMMIT_SHA	Pode esconder o RAILWAY_GIT_COMMIT_SHA automático.
OPENAI_API_KEY	O container não chama OpenAI.
RAILWAY_TOKEN	O token pertence ao job de coleta no GitHub, não à aplicação.
CORS_ORIGIN	Desnecessária quando SPA e API usam o mesmo domínio.

5.2 Teste da versão de produção

```

PRODUCTION_URL="https://SEU-DOMINIO.up.railway.app"

curl -i "$PRODUCTION_URL/api/health"
curl -s "$PRODUCTION_URL/api/version" | jq
curl -i "$PRODUCTION_URL/api/report"

```

Checkpoint de produção

Os endpoints de health, version e report devem responder corretamente antes de qualquer cenário de falha.

5.3 Criar o projeto e autorizar o repositório

1. Entre no Railway usando a conta vinculada ao GitHub.

2. Crie um novo projeto a partir de um repositório GitHub e selecione ``pipeline-guardian-agent-demo``.
3. Se o repositório da organização não estiver disponível, abra a configuração do GitHub App e conceda acesso somente ao repositório necessário para a aula.
4. Confirme que o Railway criou um serviço para a aplicação.
5. Mantenha o nome do serviço simples e estável, por exemplo ``copa-figurinhas``, pois esse valor será reutilizado no GitHub Actions.

5.4 Configurar Source, branch e build

No serviço do Railway, abra Settings e confira:

Item	Valor esperado
Source repository	<code>`pipeline-guardian-agent-demo`</code>
Branch	<code>`main`</code>
Root Directory	Vazia, utilizando a raiz do repositório.
Builder	Dockerfile
Dockerfile	Arquivo <code>`Dockerfile`</code> da raiz do projeto.
Health check path	<code>`/api/health`</code>

O ``railway.json`` e o Dockerfile já fazem parte da infraestrutura do laboratório. Não substitua o builder por outro mecanismo apenas para contornar uma falha de Docker, pois isso esconderia exatamente o tipo de problema que o cenário 4 pretende demonstrar.

5.5 Primeiro deployment de produção

Depois de confirmar a branch ``main`` e o builder, aplique as alterações e aguarde o deployment terminar.

No painel do deployment, confira separadamente:

Sinal	O que esperar
Build	Imagem construída com sucesso.
Deploy	Container iniciado e mantido em execução.
Health check	<code>`/api/health`</code> aprovado.
Runtime	Aplicação sem reinícios contínuos ou erro fatal de inicialização.

Um health check aprovado comprova que o processo está vivo. Ele não substitui o teste da rota funcional ``/api/report``, que será validada pelo smoke test.

5.6 Gerar o domínio público e registrar APP_BASE_URL

Abra:

Railway → serviço → Settings → Networking → Public Networking → Generate Domain

Copie a URL completa, incluindo ``https://``, e valide:

```
PRODUCTION_URL="https://SEU-DOMINIO.up.railway.app"
```

```
curl -i "$PRODUCTION_URL/api/health"
curl -s "$PRODUCTION_URL/api/version" | jq
curl -i "$PRODUCTION_URL/api/report"
```

Depois, volte ao GitHub e atualize `APP\_BASE\_URL` com exatamente esse domínio.

5.7 Criar o Project Token de production

No Railway, crie um token de projeto vinculado ao environment `production`:

```
Railway → Project Settings → Tokens → New Project Token
Environment: production
Name sugerido: github-actions-production
```

Copie o valor somente no momento da criação e salve-o no GitHub como secret `RAILWAY\_TOKEN`. Não cadastre esse token nas Variables do container Railway.

Escopo do token

O Project Token é associado a um environment. Um token de `production` não deve ser reutilizado como se tivesse acesso ao `staging-lab`.

5.8 Vincular o contexto Railway ao GitHub Actions

No GitHub, confira novamente as Variables relacionadas ao Railway:

Nome	Valor na produção
RAILWAY_PROJECT	Nome ou ID do projeto.
RAILWAY_SERVICE	Nome exato do serviço.
RAILWAY_ENVIRONMENT	`production`
APP_BASE_URL	Domínio público do serviço em produção.

Esses valores permitem que o workflow saiba qual aplicação consultar e qual URL testar. O `RAILWAY\_TOKEN`, por ser credencial, permanece em Secrets.

5.9 Validar a correlação da versão publicada

Acesse `/api/version` e compare o commit retornado com o commit que o Railway publicou. O laboratório prioriza `RAILWAY\_GIT\_COMMIT\_SHA`; por isso, não cadastre `COMMIT\_SHA` manualmente.

Uma validação mínima antes dos cenários deve resultar em:

Verificação	Resultado esperado
`/api/health`	HTTP 200
`/api/report`	HTTP 200
`/api/version`	Commit correspondente à release da `main`.
GitHub `APP_BASE_URL`	Mesmo domínio validado manualmente.
GitHub `RAILWAY_ENVIRONMENT`	`production`

5.10 Checklist visual antes de iniciar os cenários

- GitHub Actions está habilitado e o workflow `CI` aparece na aba Actions.
- `OPENAI\_API\_KEY` está em Secrets ou o fallback foi previamente validado.
- `RAILWAY\_TOKEN` é um token do environment `production`.
- `APP\_BASE\_URL` contém a URL pública completa com `https://`.
- Railway usa `main`, Root Directory vazia, builder Dockerfile e health check `/api/health`.
- Não existem `COMMIT\_SHA`, `NODE\_ENV`, `OPENAI\_API\_KEY` ou `RAILWAY\_TOKEN` nas variáveis do container.
- `/api/health`, `/api/version` e `/api/report` respondem corretamente.

Quando criar o staging-lab?

Somente no cenário 5. Nessa etapa, duplique `production`, aponte o environment para a branch de demonstração e crie um Project Token específico para o staging.

6. Regra de branches para os cenários

Cada cenário deve começar novamente na `main`. Não crie uma falha de testes a partir da branch de lint, nem uma falha de Docker a partir da branch de testes. Caso contrário, o pipeline encontrará primeiro a falha herdada e o relatório ficará incorreto.

```
main
├── demo/ci-lint-failure
├── demo/ci-tests-failure
├── demo/ci-docker-failure
└── demo/cd-functional-failure
```

Regra de segurança

As Pull Requests de demonstração devem ser fechadas sem merge. Depois de analisar o resultado, volte para a `main`, apague a branch temporária local e remota e só então inicie o próximo cenário.

7. Cenário 1 — Execução saudável

Neste cenário, você observará o fluxo esperado de uma entrega sem falhas. Faça uma alteração legítima e pequena, valide localmente, abra uma Pull Request e integre somente depois que todos os gates estiverem verdes.

7.1 Criar uma alteração saudável

```
git switch main
git pull --ff-only
git switch -c lab/alteracao-saudavel
```

Faça uma alteração visível, por exemplo, no cabeçalho da interface ou em um texto (sugestão: pipeline-guardian-agent-demo/frontend/index.html). Depois:

```
npm run lint
npm run test
npm run build

git add -A
git commit -m "lab: alteracao saudavel"
git push -u origin lab/alteracao-saudavel
```

Abra a PR para `main`, acompanhe o CI e faça merge somente quando os jobs técnicos estiverem aprovados.

7.2 Resultado esperado

Etapa	Resultado
Lint	SUCCESS
Testes	SUCCESS
Build	SUCCESS
Docker build	SUCCESS
CI Gate	SUCCESS
Railway Build/Deploy	SUCCESS
Health e rota funcional	SUCCESS
Smoke test e CD Gate	SUCCESS
firstFailedStage	null

Interpretação do resultado — cenário de sucesso

Um pipeline verde não significa que nenhuma linha de erro apareceu. Durante a troca de instâncias, o smoke test pode registrar falhas transitórias até que o novo container responda.

A aprovação ocorre porque a política de tentativas encontrou a versão correta, o health check passou, a rota funcional respondeu e o commit implantado correspondeu ao esperado.

O relatório saudável deve ser curto: não existe motivo para inventar causa provável ou recomendação quando não há falha técnica.

Perguntas para reflexão

- Por que uma tentativa inicial com erro não reprova automaticamente o deployment?
- Qual evidência demonstra que a versão testada é a mesma que foi implantada?

8. Cenário 2 — Falha de lint

Neste cenário, criaremos uma variável declarada e não utilizada. A aplicação não precisa estar funcionalmente quebrada para que o gate de qualidade reprove a entrega.

8.1 Criar a branch

```
git switch main
git pull --ff-only
git switch -c demo/ci-lint-failure
```

8.2 Aplicar a alteração

Abra `backend/src/services/report.js` e adicione no início de `buildReport`:

```
const totalDuplicates = stickers.length;
```

Não utilize essa variável em outro trecho. O objetivo é provocar `no-unused-vars`.

8.3 Validar, enviar e abrir a PR

```
npm run lint # deve falhar

git add backend/src/services/report.js
git commit -m "demo: viola no-unused-vars no backend"
git push -u origin demo/ci-lint-failure
```

Abra a PR para `main` e não faça merge.

8.4 Resultado esperado

Etapas	Resultado
Lint	FAILURE
Testes	SUCCESS, se não houver outro problema
Build	SUCCESS
Docker build	SUCCESS
CI Gate	FAILURE
firstFailedStage	ci_quality
Railway e CD	NOT REACHED

Interpretação do resultado — falha de lint

O lint avalia regras estáticas e padrões de qualidade. Ele pode reprovar um código que aparentemente executaria, porque a equipe decidiu que determinadas práticas não são aceitáveis.

Testes, build e Docker rodam em paralelo com o lint. Por isso, eles podem aparecer como SUCCESS. O relatório deve representar o que realmente aconteceu, e não fingir que esses jobs não foram executados.

O deployment não começa porque o CI Gate reúne os resultados técnicos e reprova a entrega antes da publicação.

Perguntas para reflexão

- Um código que executa pode ser considerado inadequado? Por quê?
- Por que os jobs paralelos não devem aparecer como NOT REACHED?

8.5 Encerrar o cenário

Feche a PR sem merge e execute:

```
git switch main
git pull --ff-only
git branch -D demo/ci-lint-failure
git push origin --delete demo/ci-lint-failure
```

9. Cenário 3 — Falha de testes

Agora o código continuará atendendo às regras do lint, mas um teste terá uma expectativa incorreta. Isso permite separar qualidade estática de comportamento funcional.

9.1 Criar a branch

```
git switch main
git pull --ff-only
git switch -c demo/ci-tests-failure
```

9.2 Alterar a expectativa

Em `backend/test/report.test.js`, altere:

```
- expect(report.duplicateCopies).toBe(1);
+ expect(report.duplicateCopies).toBe(2);
```

9.3 Validar, enviar e abrir a PR

```
npm run lint # deve passar
npm run test # deve falhar

git add backend/test/report.test.js
git commit -m "demo: expectativa incorreta para falha de testes"
git push -u origin demo/ci-tests-failure
```

Abra a PR e não faça merge.

9.4 Resultado esperado

Etapa	Resultado
Lint	SUCCESS
Testes	FAILURE
Build	SUCCESS
Docker build	SUCCESS
CI Gate	FAILURE
firstFailedStage	ci_tests

Interpretação do resultado — falha de testes

O lint passou porque o código continua sintaticamente organizado e atende às regras estáticas. Os testes falharam porque o comportamento observado não correspondeu à expectativa registrada.

Essa diferença mostra que “estar bem escrito” e “fazer o que deveria fazer” são critérios diferentes.

O relatório deve citar a expectativa e o valor observado. A explicação é mais útil quando aponta exatamente qual contrato foi violado.

Perguntas para reflexão

- O que o lint não consegue provar sobre o comportamento da aplicação?
- Por que uma expectativa de teste incorreta também deve reprovar o pipeline?

9.5 Encerrar o cenário

```
git switch main
git pull --ff-only
git branch -D demo/ci-tests-failure
git push origin --delete demo/ci-tests-failure
```

Feche a PR sem merge.

10. Cenário 4 — Falha no build da aplicação

Neste cenário, o lint e os testes devem passar, mas o processo de build do frontend deve falhar. O objetivo é mostrar que um código pode respeitar regras estáticas e ter testes aprovados, mas ainda assim não ser possível gerar o artefato de produção.

Objetivo didático

Identificar uma falha de build convencional, compreender o papel do Vite e observar como o CI registra a evidência antes da explicação gerada pelo Pipeline Guardian.

10.1 O que significa build neste projeto?

No frontend, o build transforma o código-fonte em arquivos preparados para produção. O comando da raiz aciona o workspace do frontend, que utiliza o Vite para resolver módulos, processar os arquivos e gerar a pasta **frontend/dist**.

```

Código-fonte do frontend
↓
vite build
↓
frontend/dist/
├── index.html
└── assets/
    
```

Neste cenário, não será necessário alterar o Dockerfile nem usar o Docker build como causa da falha. O foco é exclusivamente verificar se a aplicação consegue ser compilada para produção.

Comando	O que verifica
npm run lint	Verifica regras estáticas e padrões do código.
npm run test	Executa os testes automatizados e compara comportamento observado com o esperado.
npm run build	Executa o build do frontend e tenta gerar frontend/dist.

10.2 Criar a branch

```
git switch main
git pull --ff-only
git switch -c demo/ci-build-failure
```

Confirme que você está na branch correta:

```
git branch --show-current
```

10.3 Confirmar a baseline antes de implantar a falha

Antes de alterar o código, execute os três comandos. Todos devem passar. Isso evita confundir uma falha preexistente com o cenário controlado.

```
npm run lint
npm run test
npm run build
```

Checkpoint da baseline

Prossiga somente se lint, testes e build estiverem em SUCCESS. Se algum deles já falhar, corrija a main antes de continuar.

10.4 Criar uma falha controlada no build

Abra o arquivo `frontend/index.html` e localize a entrada principal do Vite:

```
<script type="module" src="/src/main.jsx"></script>
```

Altere propositalmente para um arquivo que não existe:

```
<script type="module" src="/src/main-build.jsx"></script>
```

Não crie `frontend/src/main-build.jsx`. A intenção é fazer o Vite encontrar uma referência válida do ponto de vista do HTML, mas impossível de resolver durante o build.

Por que usar o index.html?

Os testes de componentes normalmente não dependem diretamente do arquivo de entrada HTML, enquanto o Vite usa esse arquivo para iniciar o build. Assim, o cenário tende a manter lint e testes aprovados e concentrar a falha no build.

10.5 Executar os testes do cenário

Etapa A — Lint

```
npm run lint
```

Resultado esperado: **SUCCESS**. O lint verifica regras estáticas do JavaScript/JSX. A alteração está no arquivo HTML e não deve criar uma infração de lint.

Etapa B — Testes automatizados

```
npm run test
```

Resultado esperado: **SUCCESS**. Os testes continuam validando os comportamentos cobertos pela suíte. Eles não garantem que o ponto de entrada utilizado pelo bundler possa ser resolvido.

Etapa C — Build da aplicação

```
npm run build
```

Resultado esperado: **FAILURE**. O Vite inicia o build a partir do HTML, tenta localizar `/src/main-build.jsx` e encerra com exit code diferente de zero porque o arquivo não existe.

A mensagem exata pode variar conforme a versão do Vite. Procure pela evidência relacionada a `main-build.jsx`, **Could not resolve** ou outra indicação equivalente de entrada/módulo não encontrado.

Leitura do cenário

Lint responde: “o código respeita as regras?”. Testes respondem: “os comportamentos cobertos continuam corretos?”. Build responde: “consigo transformar o código atual em um artefato de produção?”. As três perguntas são diferentes.

10.6 Conferir o diff, criar commit e abrir a PR

```
git diff
```

O diff esperado é pequeno e deve mostrar apenas a mudança do ponto de entrada:

```
- <script type="module" src="/src/main.jsx"></script>
+ <script type="module" src="/src/main-build.jsx"></script>
```

```
git add frontend/index.html
git commit -m "demo: entrada inexistente para falha de build"
git push -u origin demo/ci-build-failure
```

Abra uma Pull Request para **main** e **não faça merge**.

11. Cenário 5 — Falha funcional pós-deployment

Este cenário demonstra um problema que os gates anteriores não identificaram. O código passa pelo CI, a imagem é construída, o container inicia e o health check responde. A falha aparece somente quando uma funcionalidade real é utilizada.

Objetivo técnico

Obter `/api/health` → 200 e `/api/report` → 500 na mesma versão. Isso comprova que o processo está vivo, mas uma regra de negócio está quebrada.

11.1 Criar a branch

```
git switch main
git pull --ff-only
git switch -c demo/cd-functional-failure
```

11.2 Criar uma falha controlada

Em `backend/src/routes/report.js`, deixe a função `defaultBuild` desta forma:

```
function defaultBuild() {
  const stickers = listStickers();

  if (process.env.DEMO_FUNCTIONAL_FAILURE === 'true') {
    // `stats` não existe. A rota funcional responderá HTTP 500.
    return stickers[0].stats.quantity;
  }

  return buildReport(stickers);
}
```

A falha só será ativada no environment de laboratório. Como a variável não existe localmente nem no CI, lint, testes, build e Docker continuam passando.

11.3 Validar localmente

```
npm run lint
npm run test
npm run build
docker build -t copa-figurinhas:cd-functional-failure .
```

Opcionalmente, prove a falha no container local:

```
docker run --rm -p 3001:3001 -e DEMO_FUNCTIONAL_FAILURE=true copa-figurinhas:cd-functional-failure
```

Em outro terminal:

```
curl -i http://localhost:3001/api/health
curl -i http://localhost:3001/api/report
```

11.4 Commit, push e Pull Request

```
git add backend/src/routes/report.js
git commit -m "demo: adiciona falha funcional controlada para staging"
```

```
git push -u origin demo/cd-functional-failure
```

Abra a PR para `main`, confirme que o CI está verde e não faça merge. Copie o SHA completo:

```
git rev-parse HEAD
```

Importante

Use o SHA retornado pelo seu repositório. O valor mostrado em exemplos anteriores não deve ser reutilizado em uma nova execução.

11.5 Criar o environment staging-lab

1. No Railway, abra o seletor de environments.
2. Escolha New Environment.
3. Nomeie como `staging-lab`.
4. Selecione Duplicate Environment → production.
5. Clique em Create Environment.

```
production
└─ copa-figurinhas → main

staging-lab
└─ copa-figurinhas → demo/cd-functional-failure
```

11.6 Configurar branch e variável

Com `staging-lab` selecionado:

```
copa-figurinhas → Settings → Source
Branch: demo/cd-functional-failure
```

Em Variables, adicione:

```
DEMO_FUNCTIONAL_FAILURE=true
```

Remova, caso tenham sido herdadas:

```
COMMIT_SHA
NODE_ENV
CORS_ORIGIN
```

Não preencha COMMIT_SHA manualmente

O Railway injeta `RAILWAY\_GIT\_COMMIT\_SHA` automaticamente. Um valor manual como `local` ou um hash fixo impede a automação e pode provocar uma falsa divergência de versão.

Clique em Deploy ou Apply changes. Alterar a branch ou uma variável sem aplicar as mudanças mantém o container anterior em execução.

11.7 Gerar e testar o domínio

```
Settings → Networking → Public Networking → Generate Domain
```

Defina a variável local incluindo o protocolo:

```
STAGING_URL="https://copa-figurinhas-staging-lab.up.railway.app"
```

Valide:

```
curl -s "$STAGING_URL/api/version" | jq
curl -i "$STAGING_URL/api/health"
curl -i "$STAGING_URL/api/report"
```

Endpoint	Resultado esperado
/api/version	Commit da branch de demonstração
/api/health	HTTP 200
/api/report	HTTP 500

11.8 Criar o token do staging

Railway → Project Settings → Tokens → New Project Token
 Environment: staging-lab
 Name: github-actions-staging-lab

No GitHub, substitua temporariamente o secret `RAILWAY\_TOKEN` pelo token do staging. O token de produção pode consultar o environment errado e fazer o relatório correlacionar logs de outra release.

11.9 Apontar o smoke test para o staging

Guarde a URL de produção. Depois, altere temporariamente a repository variable:

Settings → Secrets and variables → Actions → Variables
 APP_BASE_URL=https://DOMINIO-DO-STAGING.up.railway.app

11.10 Executar Post-deploy validation

Actions → Post-deploy validation → Run workflow
 Use workflow from: main
 Commit, tag ou branch a validar: SHA completo de git rev-parse HEAD
 Serviço do Railway: copa-figurinhas
 Ambiente do Railway: staging-lab

O campo “Use workflow from” indica onde está o YAML do workflow. O campo “Commit, tag ou branch a validar” indica a versão implantada que será testada.

11.11 Resultado esperado

Etapa	Resultado
Correlação do commit	MATCHED
Railway Build	SUCCESS
Railway Deploy	SUCCESS
Inicialização	SUCCESS
Health check	SUCCESS
Rota funcional	FAILURE — primeira falha
Smoke test	FAILURE
CD Gate	FAILURE
firstFailedStage	functional

Interpretação do resultado — falha funcional pós-deployment

O Railway conseguiu construir a imagem, iniciar o container e aprovar o health check. Portanto, a infraestrutura básica e o processo estavam disponíveis.

A rota `/api/report` retornou HTTP 500 porque uma regra executada somente em runtime tentou acessar uma propriedade inexistente. O problema não era “a aplicação não subiu”; era “uma funcionalidade subiu quebrada”.

O smoke test relacionou a evidência externa — HTTP 500 — ao evento interno `functional.report.failed` e ao TypeError nos logs do Railway.

O CD Gate reprovou deterministicamente. Mesmo que a IA dissesse que estava tudo bem, o resultado continuaria reprovado.

Em um disparo manual, os jobs do CI podem aparecer como UNKNOWN porque o workflow não possui um run anterior automaticamente vinculado. A PR verde é a evidência separada do CI.

Perguntas para reflexão

- Por que `/api/health` em 200 não é suficiente para afirmar que a aplicação funciona?
- Qual diferença existe entre disponibilidade do processo e disponibilidade da funcionalidade?
- O que aconteceria se o modelo gerasse uma explicação incorreta?

11.12 Restaurar o ambiente

1. Restaure `APP\_BASE\_URL` para o domínio de produção.
2. Restaure `RAILWAY\_TOKEN` para o token de produção.
3. No `staging-lab`, remova `DEMO\_FUNCTIONAL\_FAILURE` e volte a branch para `main`, ou exclua o environment.
4. Feche a PR sem merge.
5. Apague a branch somente depois que o Railway deixar de apontar para ela.

```
git switch main
git pull --ff-only
git branch -D demo/cd-functional-failure
git push origin --delete demo/cd-functional-failure
```

12. Fixtures offline e plano de contingência

Antes da aula, execute os cinco cenários sem depender de GitHub, Railway, OpenAI ou internet. As fixtures produzem relatórios equivalentes e funcionam como plano de contingência.

```
npm run pipeline:fixture -- success
npm run pipeline:fixture -- ci-lint-failure
npm run pipeline:fixture -- ci-tests-failure
npm run pipeline:fixture -- ci-docker-failure
npm run pipeline:fixture -- cd-functional-failure
```

Abra o relatório:

```
xdg-open reports/pipeline-execution-report.html
```

Como usar em aula

Caso um serviço externo esteja indisponível, apresente a fixture e peça que a turma identifique a primeira falha, as evidências, a causa provável, as limitações e o gate responsável pela decisão.

13. Solução de problemas

Sintoma	Causa provável	Correção
HTTP 301 nos curls	STAGING_URL sem https://.	Inclua o protocolo na variável.
/api/report retorna 200 no cenário 5	Branch, variável ou deployment ainda não aplicados.	Confirme staging-lab, branch, DEMO_FUNCTIONAL_FAILURE=true e clique em Deploy.
/api/version mostra local	COMMIT_SHA manual.	Exclua COMMIT_SHA e redeploy.
/api/version mostra development	NODE_ENV manual herdado.	Exclua NODE_ENV.
Relatório mostra mismatch com produção	RAILWAY_TOKEN pertence a production.	Crie e use token de staging-lab.
CI aparece UNKNOWN no cenário 5	workflow_dispatch não tem run anterior vinculado.	Use a PR verde como evidência de CI.
usedFallback=true	Sem chave ou chamada OpenAI incompleta.	O gate permanece válido; analise a categoria de erro.
src refspec ... does not match	Nome da branch no push não corresponde à branch atual.	Use git branch --show-current e git push.
env -u ainda usa OpenAI	dotenv recarregou a chave do .env.	Use OPENAI_API_KEY= vazia.

14. Checklist final e síntese

14.1 Checklist da baseline

- `npm run lint` passa na `main`.
- `npm run test` passa na `main`.
- `npm run build` passa na `main`.
- `npm run ci` passa na `main`.
- A imagem Docker da baseline é construída.
- `git status --short` não apresenta alterações.

14.2 Checklist dos ambientes

- `APP\_BASE\_URL` aponta para o ambiente correto.
- `RAILWAY\_TOKEN` pertence ao environment correto.
- Produção usa a branch `main`.
- Produção não possui `COMMIT\_SHA`, `NODE\_ENV`, `OPENAI\_API\_KEY` ou `RAILWAY\_TOKEN` nas variáveis do container.
- O `staging-lab` foi restaurado ou excluído após a demonstração.

14.3 Resultados esperados dos cinco cenários

Cenário	Primeira falha	Gate final
1. Execução saudável	Nenhuma	CD aprovado
2. Falha de lint	ci_quality	CI reprovado
3. Falha de testes	ci_tests	CI reprovado

Cenário	Primeira falha	Gate final
4. Falha no Docker build	ci_docker	CI reprovado
5. Falha funcional no CD	funcional	CD reprovado

Síntese da aula

Cenários 2 a 4: a falha ocorre antes do deployment e o CI Gate aprova.

Cenário 5: o deployment e o health check passam, mas uma funcionalidade falha e o CD Gate reprova.

Em todos os cenários, a regra técnica decide e a IA explica.

14.4 Atividade de encerramento

Escolha um dos cinco relatórios e responda:

1. Qual foi a primeira etapa que falhou?
2. Qual evidência literal comprova a falha?
3. O que é fato e o que é inferência no diagnóstico?
4. Qual gate tomou a decisão final?
5. O resultado mudaria caso o modelo produzisse outra explicação?
6. Que ação você executaria antes de tentar uma nova entrega?

15. Desafio em grupos — injeção controlada e investigação cruzada

Versão atualizada: foco em lint, testes e build da aplicação. O desafio não depende de Docker build nem de Railway.

A turma será dividida em dois grupos. Em cada rodada, um grupo introduzirá uma única falha controlada em uma branch própria. O outro grupo deverá descobrir o problema utilizando o GitHub Actions, os logs, os artifacts, o Summary e o diagnóstico do Pipeline Guardian. Na segunda rodada, os papéis serão invertidos.

Objetivo do desafio

Praticar o fluxo completo: provocar uma falha segura, observar os sinais técnicos, localizar a primeira etapa que falhou, separar evidência de hipótese, interpretar o diagnóstico da IA e confirmar a correção com uma nova execução saudável.

15.1 Preparação antes da primeira rodada

Antes de iniciar o desafio, cada grupo deve partir de uma cópia funcional do repositório e confirmar que a baseline está saudável.

```
git clone <URL_DO_REPOSITORIO>
cd pipeline-guardian-agent-demo
npm ci
npm run lint
```

```
npm run test
npm run build
```

Os três comandos de validação devem terminar com sucesso antes de qualquer falha proposital ser criada.

Importante

Neste desafio, não execute docker build e não configure RAILWAY_TOKEN. O objetivo é investigar somente falhas de CI em Quality, Tests e Build.

15.2 Regras de segurança

- Cada rodada começa novamente a partir da main saudável.
- A falha deve ser criada somente em branch de desafio. Nunca faça push direto na main.
- Nenhuma Pull Request com falha intencional deve ser integrada.
- É proibido inserir secrets, tokens, credenciais ou dados pessoais no código, nos logs ou nos comentários.
- É proibido alterar .github/workflows/ci.yml, o CI Gate, o sanitizer, o mecanismo de redaction ou a política para esconder ou forçar um resultado.
- É proibido introduzir mais de uma falha proposital na mesma rodada.
- O grupo que cria a falha não revela ao outro grupo qual arquivo, categoria ou alteração foi usada.
- O grupo investigador deve começar pelo pipeline e pelas evidências. A aba Files changed só deve ser aberta depois que uma hipótese tiver sido registrada.

15.3 Falhas permitidas

Para a atividade principal, utilize apenas uma das três categorias abaixo. Todas podem ser reproduzidas sem Railway e sem alterar o Dockerfile.

Categoria	Exemplo seguro	Validação local	Resultado esperado no CI
Lint	Declarar uma variável e não utilizá-la.	npm run lint deve falhar.	Quality falha; firstFailedStage = ci_quality.
Testes	Alterar uma expectativa existente para um valor incorreto. Ex.: duplicateCopies esperado de 1 para 2.	npm run test deve falhar; lint e build devem permanecer válidos quando aplicável.	Tests falha; firstFailedStage = ci_tests.
Build	Em frontend/index.html, alterar /src/main.jsx para /src/main-build.jsx sem criar o arquivo.	npm run lint e npm run test passam; npm run build falha.	Build falha; firstFailedStage = ci_build. O Docker build pode também aparecer vermelho como efeito em cascata, pois o Dockerfile recompila o frontend; ele não é a causa primária.

Como interpretar o cenário de Build

Se Build e Docker build ficarem vermelhos na mesma execução, registre Build como a primeira falha. O job Docker pode repetir a compilação do frontend dentro da imagem e, por isso, reproduzir o mesmo erro. A causa raiz continua sendo a impossibilidade de construir a aplicação.

15.4 Rodada 1 — Grupo A injeta, Grupo B investiga

Grupo A — preparação da falha

1. Atualize a main e confirme que ela está saudável.
2. Crie a branch challenge/round-1-failure-a.
3. Escolha uma das três categorias permitidas: lint, testes ou build.
4. Faça somente a alteração mínima necessária para produzir a falha.
5. Execute localmente os comandos necessários para comprovar que o cenário está correto.
6. Faça commit, push e abra uma Pull Request para main sem fazer merge.
7. Entregue ao Grupo B somente o link da Pull Request. Não explique a causa nem indique o arquivo alterado.

```
git switch main
git pull --ff-only
git switch -c challenge/round-1-failure-a

# depois da alteração controlada
git add .
git commit -m "challenge: falha controlada rodada 1"
git push -u origin challenge/round-1-failure-a
```

Grupo B — investigação

1. Abra primeiro a execução do GitHub Actions e a aba Summary. Não comece pelo diff do código.
2. Identifique qual job técnico falhou e registre firstFailedStage.
3. Abra o artifact ou log correspondente e copie a evidência literal que comprova a falha.
4. Leia o diagnóstico do Pipeline Guardian e classifique o conteúdo em fato, inferência, limitação e recomendação.
5. Formule uma hipótese de causa provável antes de abrir Files changed.
6. Abra o diff e o código, confirme ou rejeite a hipótese e determine a correção mínima.
7. Crie challenge/round-1-fix-b a partir da branch com falha e aplique a correção.
8. Execute npm run lint, npm run test e npm run build. Ao final, execute npm run ci como validação consolidada.
9. Faça push da correção e confirme no GitHub Actions que os gates esperados voltaram ao estado saudável.

```
git switch challenge/round-1-failure-a
git switch -c challenge/round-1-fix-b

# após corrigir
npm run lint
```

```
npm run test
npm run build
npm run ci

git add .
git commit -m "challenge: corrige falha da rodada 1"
git push -u origin challenge/round-1-fix-b
```

15.5 Rodada 2 — Grupo B injeta, Grupo A investiga

Repita o procedimento com os papéis invertidos. A segunda rodada deve começar novamente a partir da main saudável, sem herdar a falha da rodada anterior.

```
challenge/round-2-failure-b
challenge/round-2-fix-a
```

O Grupo B escolhe uma nova categoria de falha controlada. O Grupo A executa toda a investigação, explicação, confirmação da causa e correção. Preferencialmente, não repita a mesma categoria utilizada na primeira rodada.

15.6 Entrega do grupo investigador

Item	O que deve ser apresentado
Primeira falha	Job técnico e firstFailedStage.
Evidência	Trecho literal do log ou resultado técnico que comprova o problema.
Explicação da IA	Resumo do que o Pipeline Guardian interpretou ou, em fallback, do diagnóstico determinístico.
Fato x inferência	O que veio diretamente das evidências e o que foi hipótese do diagnóstico.
Gate	Qual gate reprovou a entrega e por qual regra técnica.
Causa confirmada	Arquivo e trecho encontrado somente após a investigação.
Correção	Mudança mínima aplicada pelo grupo investigador.
Validação final	Nova execução com lint, testes, build e CI em estado esperado.

15.7 Critério de sucesso

O desafio não termina quando o grupo “adivinha” a falha. Ele termina quando existe uma cadeia de evidências reproduzível:

Cadeia de evidências

falha observada → job identificado → evidência coletada → explicação analisada → hipótese formulada → causa confirmada → correção aplicada → nova execução saudável

O grupo investigador deve provar por que a falha aconteceu. A explicação da IA ajuda a organizar o raciocínio, mas a comprovação vem dos resultados técnicos, dos logs, do código e da nova execução.

15.8 Preparação da IA para o desafio

O RAILWAY_TOKEN não é necessário nesta atividade, porque o desafio principal termina no CI e utiliza somente lint, testes e build.

A OPENAI_API_KEY também não é obrigatória para o funcionamento do Pipeline Guardian: sem a chave, o projeto deve continuar utilizando o fallback determinístico. Se o professor desejar demonstrar a explicação por modelo real, a chave deve ser cadastrada como Secret do GitHub Actions e nunca inserida no código, no .env versionado, em Variables ou em mensagens da Pull Request.

GitHub → Settings → Secrets and variables → Actions → Secrets → New repository secret

Name: OPENAI_API_KEY

Secret: <valor cadastrado pelo professor>

Orientação de segurança para a turma

Não distribua a chave em texto puro. Para uma prática com modelo real, prefira uma chave/projeto exclusivo para a aula, com uso monitorado, e revogue a credencial ao final. Se os grupos trabalharem em forks, o Secret do repositório original não é disponibilizado automaticamente ao workflow da Pull Request; nesse caso, use o fallback ou prepare previamente um repositório controlado para cada grupo.

15.9 Checklist rápido do professor

- Main saudável antes da aula.
- Node.js 22 disponível nas máquinas.
- Cada grupo consegue clonar o repositório e executar npm ci.
- Cada grupo possui um repositório ou permissão para criar branch, push e Pull Request.
- GitHub Actions habilitado.
- OPENAI_API_KEY cadastrada somente se a execução com LLM real for desejada.
- RAILWAY_TOKEN não é necessário.
- Categorias do desafio limitadas a lint, testes e build.
- Nenhuma branch com falha intencional será mergeada na main.